Drivique: diseño y especificación de una plataforma web progresiva y aplicación móvil para optimizar la gestión del alquiler de vehículos en Colombia

Laura Vanessa Pérez Perdomo
Tecnólogo en Análisis y Desarrollo de Software
SENA – Centro de la Industria, la Empresa y los Servicios
Ficha 3145555 · Colombia
p.laurav2@gmail.com

Danna Valentina Barrios Penagos
Tecnólogo en Análisis y Desarrollo de Software
SENA – Centro de la Industria, la Empresa y los Servicios
Ficha 3145555 · Colombia
dannabarrios0820@gmail.com

Emily Sharith Amézquita Saavedra
Tecnólogo en Análisis y Desarrollo de Software
SENA – Centro de la Industria, la Empresa y los Servicios
Ficha 3145555 · Colombia
sharithamezquita81@gmail.com

Abstract— Vehicle rental in Colombia still relies largely on manual processes for booking, contracting, payment and fleet control, which cause delays, errors and limited traceability. This paper presents the study and design of Drivique, a vehicle rental platform made up of a progressive web application (PWA) and an Android client, backed by a REST API with an N-tier architecture. The specification, written under the IEEE 830 standard, defines 60 functional requirements grouped into nine modules, 12 non-functional requirements and a set of use cases. The system integrates a four-step booking wizard, online payments through Wompi (cards, PSE, Nequi and Bancolombia button), cash payment with dynamic expiration, electronic contract signing, PIN-validated vehicle handover, vehicle inspection, and an administrative panel with auditing and reporting. Security relies on role-based JWT authentication, TLS 1.3 and the OWASP Top 10 guidelines, while accessibility follows WCAG 2.1. This work provides the foundation for the implementation and user validation of the solution.

***Keywords*— vehicle rental; digital transformation; PWA; REST API; N-tier architecture; Wompi; JWT; accessibility.**

Resumen— El alquiler de vehículos en Colombia aún depende, en buena medida, de procesos manuales de reserva, contratación, cobro y control de flota que generan demoras, errores y baja trazabilidad. Este artículo presenta el estudio y el diseño de Drivique, una plataforma de alquiler de vehículos compuesta por una aplicación web progresiva (PWA) y un cliente móvil Android, respaldada por una API REST con arquitectura en N capas. La especificación, elaborada bajo el estándar IEEE 830, define 60 requerimientos funcionales organizados en nueve módulos, 12 requerimientos no funcionales y un conjunto de casos de uso. El sistema integra reservas en un asistente de cuatro pasos, pagos en línea mediante Wompi (tarjetas, PSE, Nequi y Botón Bancolombia), pago en efectivo con vencimiento dinámico, firma electrónica de contratos, entrega y recogida validadas con un PIN de seis dígitos, inspección de vehículos y un panel administrativo con auditoría y reportes. La seguridad se apoya en JWT con control de acceso por roles, cifrado TLS 1.3 y los lineamientos OWASP Top 10; la accesibilidad, en WCAG 2.1. El trabajo constituye la base para la implementación y la validación con usuarios de la solución.

***Palabras claves*— alquiler de vehículos; transformación digital; PWA; API REST; arquitectura en N capas; Wompi; JWT; accesibilidad.**

I. Introducción

La digitalización de los servicios de movilidad ha dejado de ser una ventaja competitiva para convertirse en una expectativa del cliente. En Colombia, el mercado de alquiler de vehículos generó cerca de 288 millones de euros en 2024 y se proyecta que supere los 345 millones en 2029, con una tasa de crecimiento anual compuesta de 3,68 %; además, se estima que alrededor del 44 % de esos ingresos provendrá de ventas en línea [1]. Otras estimaciones, que consideran una definición más amplia del mercado, lo ubican en 1,11 miles de millones de dólares en 2024 y en 2,64 miles de millones hacia 2033 [2].

Pese a este panorama, muchas empresas del sector mantienen reservas por llamada o atención presencial, contratos en papel, cobros no integrados y un control limitado de la disponibilidad y el mantenimiento de la flota. Estas prácticas producen demoras, errores humanos, pérdida de documentos y subutilización de los vehículos. La literatura del sector sugiere que la digitalización de la operación tiene efectos medibles: se reportan reducciones de 10 % a 15 % en el costo de combustible mediante mejor planificación de rutas y monitoreo de conducción [3], y una empresa colombiana de renting informó un aumento de productividad del 200 % tras adoptar un sistema de telemática en más de 4.000 vehículos [4].

En este contexto se plantea la pregunta que orienta el trabajo: ¿cómo puede una solución tecnológica integral optimizar los procesos de alquiler de vehículos, en particular reservas, contratos, pagos, atención al cliente, accesibilidad y gestión de flota, para mejorar la experiencia del usuario y la competitividad del negocio? Como respuesta se propone Drivique, una plataforma que unifica en un solo sistema el recorrido del cliente, desde el catálogo hasta la devolución del vehículo, y la operación de la empresa y de sus sucursales.

El objetivo de este artículo es documentar el estudio y el diseño de Drivique a partir de su Especificación de Requisitos de Software (SRS). Sus contribuciones son: (i) un conjunto trazable de requerimientos funcionales y no funcionales; (ii) una arquitectura en N capas con API REST, cliente web progresivo y cliente móvil; y (iii) un flujo operativo que combina pago digital y presencial, firma electrónica y entrega validada con PIN. El resto del documento presenta la fundamentación teórica (sección II), la metodología (sección III), los resultados del diseño (sección IV) y las conclusiones (sección V).

II. Fundamentación teórica

A. Ingeniería de requisitos y estándar IEEE 830

La especificación de requisitos de software describe de forma completa, consistente y verificable lo que el sistema debe hacer. El estándar IEEE 830-1998 recomienda una estructura que incluye introducción, descripción general, requisitos específicos e interfaces [5]. Drivique adopta esa estructura y complementa cada requerimiento con identificador, tipo, entradas, salidas, acciones y criterios de aceptación.

B. Arquitectura en N capas y API REST

La arquitectura en N capas separa las responsabilidades de presentación, lógica de negocio y acceso a datos, lo que facilita el mantenimiento, las pruebas y la escalabilidad. La comunicación entre capas se realiza mediante una API REST, estilo arquitectónico basado en recursos, operaciones HTTP sin estado e intercambio de representaciones, como el formato JSON [6].

C. Aplicaciones web progresivas (PWA)

Una PWA es una aplicación web que, mediante service workers, manifiesto e HTTPS, ofrece una experiencia similar a la de una aplicación instalada: acceso desde el icono de inicio, carga rápida y diseño adaptable [7]. Para Drivique, esta opción reduce el costo de desarrollo frente a aplicaciones nativas y permite un único código base para escritorio y móviles.

D. Seguridad: JWT, control por roles y OWASP

JSON Web Token (JWT) es un formato compacto para transmitir afirmaciones firmadas entre partes [8]. En Drivique el token viaja en los encabezados HTTP y habilita un control de acceso basado en roles. Los riesgos de seguridad de las aplicaciones web se contrastan con la lista OWASP Top 10 [9], y el canal se protege con TLS 1.3 [10].

E. Pasarela de pagos Wompi

Wompi es una pasarela de pagos colombiana que ofrece un widget de pago y una API REST con notificaciones por webhook para confirmar transacciones [11]. Permite cobrar con tarjetas de crédito y débito, PSE, Nequi y Botón Bancolombia, métodos de uso extendido en el país.

F. Accesibilidad y trabajos relacionados

Las pautas WCAG 2.1 nivel AA establecen criterios de percepción, operabilidad, comprensión y robustez de los contenidos web [12]. Su importancia se ha documentado en proyectos colombianos de tecnología asistiva: Alarcón et al. encontraron que muchos usuarios con discapacidad visual o motriz de Neiva no sabían leer ni escribir, por lo que su aplicación móvil debía basarse en iconos y mantener una comunicación permanente con el usuario [13]. Drivique recoge esta lección al priorizar interfaces simples, consistentes y adaptables.

III. Metodología

El trabajo siguió un enfoque aplicado de ingeniería de software, orientado a producir una especificación y un diseño verificables antes de la implementación. Se desarrolló en cuatro fases:

Fase 1: análisis del problema. Se planteó el diagnóstico de los procesos actuales de empresas de alquiler mediante entrevistas, observación de procesos y revisión documental, y se documentó el problema, el propósito, la justificación y el alcance.

Fase 2: especificación de requisitos. Se redactó el SRS conforme a IEEE 830: perfiles de usuario, restricciones, suposiciones y dependencias, interfaces de usuario, hardware, software y comunicación, 60 requerimientos funcionales y 12 no funcionales.

Fase 3: prototipado. Se elaboraron bosquejos de las interfaces web y móvil en Figma para validar la navegación y la jerarquía visual antes de programar.

Fase 4: diseño arquitectónico y casos de uso. Se definieron las capas, los roles, las integraciones externas y los casos de uso (registro, inicio de sesión, entre otros), con sus precondiciones, secuencias normales, excepciones y postcondiciones.

Cada requerimiento funcional se redactó con un identificador (RF1 a RF60) y se agrupó en módulos, de modo que cada pantalla, regla de negocio y llamada a la API pueda trazarse hasta su origen y verificarse mediante criterios de aceptación.

IV. Resultados

A. Arquitectura propuesta

La Fig. 1 muestra la arquitectura. Los clientes web (React, PWA) y móvil (Android) consumen una API REST por HTTPS con JSON. La lógica de negocio se implementa en Spring Boot y se separa de la capa de datos, que usa un motor relacional (PostgreSQL o MySQL) con migraciones versionadas mediante Liquibase. El sistema se integra con Wompi, con un servidor SMTP para notificaciones por correo y con el widget de chat Tawk.to para soporte en tiempo real.

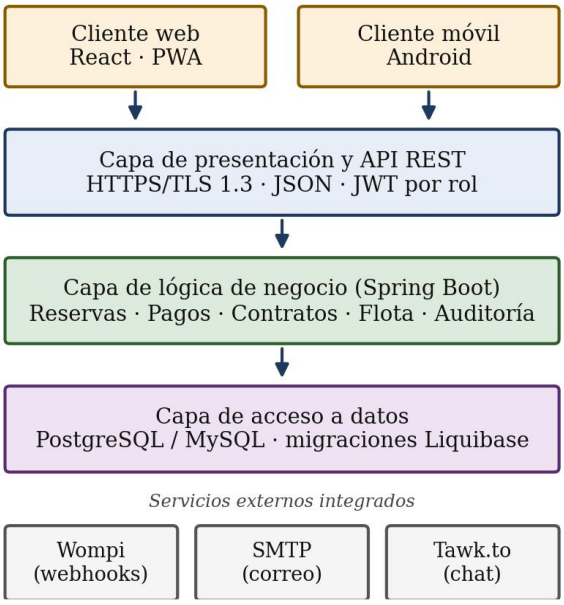


Fig. 1. Arquitectura en N capas de Drivique.

B. Roles y control de acceso

La autenticación usa JWT y la autorización se resuelve por rol (Tabla I). Tras iniciar sesión, cada usuario es redirigido a la vista que le corresponde y se bloquea la página pública cuando existe una sesión activa.

TABLA I. ROLES DE USUARIO

Rol	Funciones principales
Cliente	Consultar catálogo, reservar, pagar, firmar contratos, ver su PIN, calificar y reportar incidencias.
Encargado de sucursal	Confirmar pagos en efectivo, verificar documentos, entregar y recibir vehículos, gestionar cupones locales y responder reseñas.
Super administrador	Gestionar ciudades, sucursales, flota, usuarios, roles, promociones, reportes, auditoría e identidad visual.

C. Módulos funcionales

Los 60 requerimientos funcionales se agrupan en nueve módulos (Tabla II). El módulo 9 concentra 27 de los 60 requerimientos (45 %), lo que refleja la complejidad de la operación administrativa y de sucursal.

TABLA II. MÓDULOS Y REQUERIMIENTOS

N.º	Módulo	Requisitos
1	Página de inicio pública	RF1–RF2
2	Autenticación y gestión de cuenta	RF3–RF8
3	Catálogo y experiencia del cliente	RF9–RF18
4	Asistente de reserva (4 pasos)	RF19–RF22
5	Pasarela de pagos	RF23–RF24
6	Contratos, firma y entrega con PIN	RF25–RF27
7	Mis reservas y perfil	RF28–RF31
8	Componentes transversales	RF32–RF33
9	Panel administrativo y sucursal	RF34–RF60

D. Flujo de alquiler

La Fig. 2 resume el recorrido. El asistente de reserva se organiza en cuatro pasos: (1) fechas, horarios, sucursales, entrega a domicilio, verificación de pico y placa y modalidad de kilometraje; (2) coberturas (básica, estándar y premium) y servicios adicionales; (3) datos del conductor, licencia y cupones; y (4) resumen financiero con IVA del 19 %, depósito de garantía y método de pago. Al final se genera un código único de reserva.

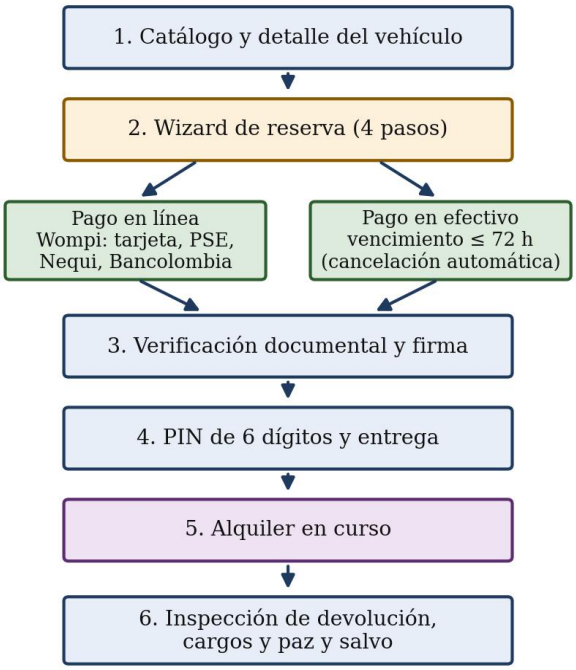


Fig. 2. Flujo operativo de un alquiler en Drivique.

El pago puede ser digital o presencial. En línea, el widget de Wompi procesa la transacción y un webhook actualiza la reserva a confirmada y pagada. En efectivo, la reserva queda pendiente con un plazo máximo de 72 horas (o hasta dos horas antes de la recogida si es de última hora); si el plazo vence, un proceso automático la cancela. El cliente carga su cédula y licencia, firma el contrato en un lienzo digital y recibe un PIN de seis dígitos que el personal de la sucursal debe validar para entregar las llaves. La firma registra marca de tiempo, dirección IP y hash criptográfico. En la devolución, el sistema compara la inspección final con la inicial y calcula cargos por kilometraje, combustible, demora o daños.

E. Requisitos no funcionales

La Tabla III sintetiza los requisitos no funcionales más relevantes. Se priorizó la seguridad de los datos personales, que se rige por la normativa colombiana de protección de datos (Habeas Data), y la continuidad del servicio ante fallas de la pasarela, para lo cual se ofrece el pago en efectivo como alternativa.

TABLA III. REQUISITOS NO FUNCIONALES CLAVE

Atributo	Meta especificada
Rendimiento	Respuesta < 2 s en peticiones comunes; hasta 1.000 req/s; confirmación de reserva < 3 s.
Seguridad	HTTPS/TLS 1.3, JWT, contraseñas con BCrypt (factor 12), Habeas Data.
Disponibilidad	99,9 % anual con redundancia y balanceo de carga.
Usabilidad	Navegación en menos de 3 clics; WCAG 2.1; modo claro y oscuro.
Compatibilidad	Android 8.0+ y diseño responsivo de 320 px a 4K.
Internacionalización	Español, inglés, francés, portugués y portugués de Brasil.
Mantenibilidad	Capas desacopladas, principios SOLID, Swagger/OpenAPI 3.0.
Resiliencia	Circuit breakers, reintentos con retroceso exponencial y recuperación de transacciones.

F. Alcance y limitaciones

El sistema no incluye facturación electrónica con validez fiscal, contabilidad avanzada ni aplicación nativa para iOS. Los pagos se procesan en pesos colombianos y se requiere conexión a internet. Estas decisiones acotan el alcance a un producto viable y verificable.

V. CONCLUSIONES Y TRABAJO FUTURO

Este trabajo presentó el estudio y el diseño de Drivique, una plataforma que integra en un único sistema el catálogo, la reserva, el pago, la contratación digital, la entrega validada con PIN, la devolución y la administración de flota. La especificación basada en IEEE 830 deja un conjunto trazable de 60 requerimientos funcionales, 12 no funcionales y casos de uso que sirve de guía para la construcción y las pruebas.

Entre las decisiones de diseño más relevantes se destacan la combinación de pago en línea y en efectivo con vencimiento dinámico, que amplía la inclusión financiera y mitiga la dependencia de la pasarela; la validación presencial con PIN y la firma electrónica con huella criptográfica, que reducen el riesgo de fraude y de disputas; y la arquitectura en N capas con control de acceso por roles, que favorece la escalabilidad y la seguridad.

Debe señalarse que los resultados corresponden a la fase de especificación y diseño: las metas de rendimiento, disponibilidad y accesibilidad aún no se han medido en una implementación en producción. Como trabajo futuro se plantea implementar el sistema, ejecutar pruebas de carga y de seguridad, evaluar la usabilidad con clientes y encargados de sucursal, y ampliar el alcance a facturación electrónica, versión para iOS y modo sin conexión.

REFERENCIAS

[1]Statista, “Car Rentals – Colombia,” Market Insights. [En línea]. Disponible: https://es.statista.com

[2]Deep Market Insights, “Colombia Car Rental Market 2025–2033.” [En línea]. Disponible: https://deepmarketinsights.com

[3]Ken Research, “Colombia Fleet Management Software Market.” [En línea]. Disponible: https://kenresearch.com

[4]Automotive Fleet, “Renting Colombia S.A. y la adopción de telemática MiX Telematics.” [En línea]. Disponible: https://automotive-fleet.com

[5]IEEE, “IEEE Recommended Practice for Software Requirements Specifications,” IEEE Std 830-1998, 1998.

[6]R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” tesis doctoral, Univ. of California, Irvine, 2000.

[7]web.dev, “Progressive Web Apps.” [En línea]. Disponible: https://web.dev/progressive-web-apps

[8]M. Jones, J. Bradley y N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.

[9]OWASP Foundation, “OWASP Top 10:2021.” [En línea]. Disponible: https://owasp.org/Top10

[10]E. Rescorla, “The Transport Layer Security (TLS) Protocol Version 1.3,” RFC 8446, IETF, 2018.

[11]Wompi, “Documentación oficial de la API.” [En línea]. Disponible: https://docs.wompi.co

[12]W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” 2018. [En línea]. Disponible: https://www.w3.org/TR/WCAG21

[13]A. H. Alarcón López, G. Arias Vargas, C. J. Cadena Sarasty y V. Muñoz Álvarez, “Estudio y diseño de un sistema domótico utilizando dispositivos móviles para mejorar la accesibilidad de las personas discapacitadas,” 3er Congreso Internacional AmITIC, Santa Cruz de la Sierra, Bolivia, 2016, pp. 45–50.